SBMS SECURITY AND ACCESS CONTROL CATALOGUE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/index.js`, `backend/src/middleware/*`, `backend/src/config/{cors,jwtSecret,publicAppUrl,settings,reconciliation}.js`, all twelve routers, `backend/src/controllers/*`, `docs/bursary-certificates-2026-11-01.md` §6, `docs/qr-verification-2026-10-30.md` §1-§5.

---

1. Summary of the control set

Control	Implemented as	Where
Password storage	bcrypt, cost factor 12	`auth.controller.js:26`
Session credential	HMAC-signed JWT, `expiresIn: 8h`, secret validated at start-up	`auth.controller.js:55-59`; `config/jwtSecret.js`
Role separation	Two sign-in endpoints; wrong endpoint refused with 403	`auth.routes.js:12-13`; `auth.controller.js:52`
Authorisation	`requireRole(...)` per router or per route; role re-read from the database on every request	`middleware/auth.js:20-24`; `middleware/requireRole.js`
Account state	`users.is_active` must be TRUE; a deleted user is refused	`middleware/auth.js:22-23`
Origin control	Configured allowlist checked before routing, plus the `cors` middleware	`index.js:40-47`; `config/cors.js`
Security headers	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy: microphone=(), geolocation=()`	`index.js:33-39`
Body bound	`express.json({limit: '100kb'})`	`index.js:64`
Error sanitation	5xx returns `Internal server error`; only allowlisted metadata is logged	`middleware/errorHandler.js:59-78`
Rate limits	10 sign-ins, 10 student verifications, 20 certificate verifications per IP per 15 minutes	`middleware/*RateLimit.js`; `rateLimitWindow.js`
Public disclosure control	`{found}` only; authenticity facts only; no identifiers	`verification.controller.js`; `certificates.controller.js:251-290`
Identifier masking	Last three characters until an audited reveal	`duplicateFlags.controller.js:37-42,106-107`
Audit trail	`audit_logs` row for every state change	`services/auditLog.js`
Start-up guards	Missing or weak `JWT_SECRET`, unsafe `PUBLIC_APP_URL`, wrong database identity, missing login columns all stop the process	`index.js:102-125`; `config/publicAppUrl.js`; `services/databaseStartup.js`

2. Authentication

Property	Value	Notes
Hash	bcrypt, cost 12	Cost factor is not benchmarked on the deployment host; a slower host would raise sign-in latency
Token	JWT, HS256, 8-hour expiry	No refresh token and no server-side revocation list; a stolen token is valid until expiry or until the account is deactivated
Token transport	`Authorization: Bearer` header	The frontend keeps the token in `localStorage` (`frontend/src/services/api.js:13`)
Secret minimum	32 bytes	Enforced at start-up; also rejected if it contains a placeholder word or has fewer than 8 distinct characters

Property	Value	Notes
Secret generation instruction	`node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"`	README; the value must never be committed
Sign-in separation	`/api/auth/login/admin` refuses a field officer with 403; `/api/auth/login/field-officer` refuses an administrator with 403	Prevents a role being used at the wrong door, though the role is drawn from the database either way
Failed sign-in	`401 Invalid email or password`	Identical for unknown email and wrong password, so the endpoint does not confirm account existence
Account enumeration at sign-up	Public registration is 410; accounts are created by an administrator	No sign-up oracle exists
Lockout	Per client IP only (10 attempts / 15 min)	No per-account lockout and no progressive delay

3. Authorisation matrix

Capability	Field Officer	Administrator	Unauthenticated
Sign in	yes (officer endpoint)	yes (admin endpoint)	yes
Register / list / read students	yes	**no (403)**	no
Compute eligibility	yes	no	no
Submit award application	yes	no	no
Record beneficiary verification	yes	no	no
Approve or reject award	no (403)	yes	no
Issue, read, print, revoke certificate	read and print only	yes (issue and revoke)	no
Record, clear, acknowledge payment	yes	yes	no
Link a legacy payment to an award	no	yes	no
Read duplicate queue, reveal identifiers, decide case	no (403)	yes	no
Rescan the registry	no	yes	no
Read/update cut-offs	no	yes	no
Dashboard, reports export, audit log, users	no	yes	no
Student QR verification	—	—	yes, rate limited, `{found}` only
Certificate verification	—	—	yes, rate limited, authenticity facts only
Health and readiness	—	—	yes

The four role boundaries proven by `backend/tests/workflow.test.js` #11 are: an officer cannot adjudicate a case (403) or approve an award (403); an administrator cannot register a student (403) or create an award (403).

4. Identity resolution and role freshness

`authenticateToken` verifies the signature, then presents `decoded.id || decoded.sub` to the database and rebuilds `req.user` from the stored row:

```
SELECT id, role FROM users WHERE id = $1 AND is_active = TRUE
```

Three consequences worth stating in the report. First, a role change takes effect on the next request without re-issuing a token, because the signed role claim is ignored. Second, deactivating an account is an immediate revocation mechanism, and it is the only one the system has. Third, a token whose subject no longer exists answers `401 User account no longer exists` rather than a 500, so a deleted account cannot be mistaken for a server fault.

5. Secrets and configuration

Secret or setting	Where it lives	Guard
`JWT_SECRET`	Environment variable, never in the repository	>= 32 bytes; placeholder patterns rejected; start-up aborts otherwise
`DATABASE_URL`	Environment variable	Database identity asserted at start-up and on `/api/ready`
`SBMS_EXPECTED_DATABASE_NAME`	Environment variable	Readiness fails closed when the connected database name differs
`PUBLIC_APP_URL`	Environment variable	Must be a public HTTPS origin in production; LAN and plain-HTTP origins rejected
`CORS_ORIGINS`	Environment variable	Comma-separated bare origins; a malformed value throws and stops the API
`TRUST_PROXY_HOPS`	Environment variable	Default 0; forwarded headers are never trusted unless set
`ACKNOWLEDGEMENT_OVERDUE_DAYS`	Environment variable	Integer 1-3650; unset disables overdue reporting rather than inventing a deadline
Deployment credentials	`deploy/azure/.secrets.json`	Git-ignored and never printed; records the administrator and JWT secrets used for the demonstration deployment

The start-up failure message deliberately appends the cause (`index.js:117-124`), because a Container Apps environment with logging disabled would otherwise show an unexplained crash loop. The comment states that the message is authored text and carries no credentials; stacks are still withheld.

6. Origin control, proxy trust and headers

Control	Behaviour	Limitation
Pre-routing origin check	A request carrying a disallowed `Origin` header is refused with 403 before any route matches	A non-browser client can omit the header; CORS is a browser policy, not authentication
`cors` middleware	Reflects only allowlisted origins	Same as above
Default allowlist	`http://localhost:4174` in development, empty in production unless configured	A missing `CORS_ORIGINS` in production denies browser origins rather than allowing all
Proxy trust	Off by default; `TRUST_PROXY_HOPS` opts in	Mis-set hops would let a client spoof `X-Forwarded-For` and evade the per-IP limiter
Security headers	`nosniff`, `DENY`, `no-referrer`, restricted permissions	**No `Content-Security-Policy` and no `Strict-Transport-Security` header is set by the API**; TLS and HSTS are properties of the hosting platform

7. Input validation and error handling

Layer	Mechanism	Coverage
Request bodies	Zod schemas per controller with cross-field refinements (evidence required for clearance or verification, amount required for approval, reason length)	Every mutating endpoint
Identifiers	Uppercased and normalised; blank strings dropped rather than stored as empty	`services/studentRecordFormat.js`
Phone numbers	Must normalise to `+254[17]` plus eight digits	`students.controller.js:23`
Dates	ISO validation in the API; day-first parsing in the ML feature builder	`studentRecordFormat.validIsoDate`; `feature_engineering._dob_similarity`
Enum values	`school_code` one of five; `role`, `decision`, `payment_status`, `outcome` constrained	Controllers and database CHECK constraints
Quantities	Amounts coerced to finite positive numbers bounded at 9 999 999 999.99; acknowledgement allows zero	Award and disbursement schemas

Layer	Mechanism	Coverage
Unparseable JSON	`Invalid request`	`errorHandler.js:73-77`
Oversize body	`Request body is too large`	`errorHandler.js:75`
Any 5xx	`Internal server error`, stack never serialised	`errorHandler.js:73-77`

8. Data protection and privacy

Property	Implementation	Consequence
QR token placement	Token in the URL **fragment** (`/verify#token=...`), removed from the visible URL with `history.replaceState`, then POSTed	The token never reaches a web server in a URL, never appears in access logs and is not sent as a `Referer`
Token format	Exactly 64 lowercase hex characters; exactly one body key accepted	Malformed, unknown and well-formed-but-unknown tokens all answer `{found: false}`, so the endpoint is not a validity oracle
Student endpoints	No name, school, date of birth, identifier or phone in any response	Verification confirms existence only, never identity, eligibility, approval or payment
Certificate endpoint	Authenticity facts only; no student name, UPI, school or amount	A holder of the QR learns that a certificate exists and is current, not whose it is
Certificate storage	Snapshot row, `award_id` UNIQUE, revocation separate from the award	A printed document cannot be rewritten by a later award edit
Review queue	Identifiers masked to the last three characters	Disclosure requires an explicit, audited request
Evidence references	Free-text pointers, not uploads	No document is stored by SBMS, so there is no document store to breach, and no document integrity guarantee either
Transport	HTTPS in deployment	Local development runs over HTTP by design
Encryption at rest	Provided by the managed PostgreSQL service, not the application	No column-level encryption for identifiers
Personal data minimisation	Only fields needed for screening, identification and payment are stored	The national identifier is stored but is never a model feature
Retention	None implemented	Audit, review and payment rows are kept indefinitely (G-19)

9. Audit trail

Property	Value
Writer	`services/auditLog.js`, called inside the same transaction as the change
Fields	`user_id`, `action`, `table_name`, `record_id`, `details` (JSONB), `created_at`
Actions recorded	`CREATE` on registration and award submission; `AWARD_VERIFICATION_RECORDED`; duplicate confirm and reject actions; `AWARD_APPROVED` / `AWARD_REJECTED`; `PAYMENT_ISSUED`, `PAYMENT_CLEARED`, `PAYMENT_FAILED`, `PAYMENT_CANCELLED`; `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`; `AWARD_CERTIFICATE_ISSUED`, `AWARD_CERTIFICATE_REVOKED`; threshold updates; identifier reveals
Screening provenance	`duplicate_model_version`, `duplicate_review_threshold`, `duplicate_aggregation_policy`, candidate counts, flags created
Ten-action sequence proven	`tests/workflow.test.js` #10
Read access	Administrator only (`GET /api/audit-logs`), rendered readably rather than as raw JSON
Immutability	**By convention only.** No trigger, no permission restriction, no append-only table, no retention policy

10. Rate limiting, and exactly how far it reaches

Limiter	Budget	Reach	Honest limitation
Sign-in	10 / IP / 15 min	Both sign-in routes share one counter map	Per IP only; a distributed attacker is not slowed, and a shared-NAT school could be locked out until the window expires
Student verification	10 / IP / 15 min	`/api/verify`	Counters are process-local: they reset on restart and are not shared between container replicas
Certificate verification	20 / IP / 15 min	`/api/certificates/verify`	Same
IP map ceiling	10 000 IPs per limiter	All three	Once full, new IPs are refused with 429 until cleanup runs

Rate limiting is mounted before the JSON parser on the two public verifiers, so an abusive client is rejected before its body is parsed.

11. Residual security risks to report

Risk	Severity for a demonstration	Reason it cannot be closed inside this project
JWT in `localStorage`	Medium	Mitigating it properly needs httpOnly cookies, CSRF handling and a same-site frontend and API origin; the frontend is served from a different host than the API
No token revocation list	Medium	Needs a store and a sign-out endpoint; the only revocation today is deactivating the account
No per-account lockout	Medium	Needs a counter keyed on the account as well as the IP
No multi-factor authentication	Medium	Out of scope for the proposal; administrator actions are high-value
In-process rate limiter	Medium	Needs a shared store to survive scaling and restarts
No CSP or HSTS header from the API	Low to medium	Static frontend hosting controls these; the API serves JSON only
No row-level security in PostgreSQL	Medium	Any client with database credentials bypasses all application control
Audit trail is not immutable	Medium	Needs a restricted role, a trigger or an append-only store
Route-scoped admin-login diagnostic logs error name, PostgreSQL code and file:line	Low	Marked temporary in the source; it logs no credential, body or stack, but it is a deliberate diagnostic left in the code
No security review or penetration test	High for any production claim	Never performed; nothing in this catalogue is a substitute
No dependency-vulnerability gate in CI	Medium	No `npm audit` or Python audit gate is part of the recorded pipeline

12. How to describe the security position in Chapters 4-6

1. State the controls that exist and are tested (§1, §3, §9, §10) together with the test that proves each.
2. State the risks in §11 explicitly, as residual risks of a demonstration system, not as future work that has been done.
3. Do not describe the system as production-ready, hardened or secure without a review; no penetration test, no dependency audit and no immutable audit store exist.
4. Keep the distinction between *authenticated* (who you are), *authorised* (what role you hold) and *accountable* (what was recorded). SBMS implements all three, and the audit trail is the weakest, because it is only conventionally immutable.